

Here's a complete Fortran subroutine that writes the CAFFA program results to **VTK Structured Grid** format (`.vts` file, which is the modern XML-based VTK format for structured grids). This is much better than the old legacy `.vtk` format.

Subroutine: `WRITE_VTK`

Add this subroutine to your `user.f` file (or anywhere in the code after the main program).

```
fortran
C#####
      SUBROUTINE WRITE_VTK(K, ICOUNT)
C#####
C    Writes CAFFA solution on grid level K to VTK Structured Grid
C    format (*.vts) for visualization in ParaView, VisIt, etc.
C
C    Called from user routines (e.g. SOUT or TOUT) when you want
C    to export results.
C=====
      INCLUDE 'param.inc'
      INCLUDE 'indexc.inc'
      INCLUDE 'var.inc'
      INCLUDE 'geo.inc'
      INCLUDE 'model.inc'
      INCLUDE 'logic.inc'

      INTEGER K, ICOUNT
      CHARACTER*80 FNAME
      CHARACTER*3  DSN
      INTEGER I, J, IJ
      REAL*8 XC_PHYS, YC_PHYS

      CALL SETIND(K)

C.....Construct filename: e.g. "case.001.vts" or "case_03.vts"
      IF (ICOUNT .LT. 10) THEN
        WRITE(DSN,'(I1,2H )') ICOUNT
      ELSE IF (ICOUNT .LT. 100) THEN
        WRITE(DSN,'(I2,1H )') ICOUNT
      ELSE
        WRITE(DSN,'(I3)') ICOUNT
      FNDTF
```

```

-----

WRITE(FNAME,'(A6,A1,A3,A4)') NAME, '_', DSN, '.vts'

OPEN(UNIT=20, FILE=FNAME, FORM='FORMATTED', STATUS='UNKNOWN')
REWIND(20)

C.....Write VTK XML StructuredGrid header
WRITE(20,'(A)') '<?xml version="1.0"?>'
WRITE(20,'(A)') '<VTKFile type="StructuredGrid" version="0.1" '//
&                'byte_order="LittleEndian"
compressor="vtkZLibDataCompressor">'
WRITE(20,'(A,I5,A,I5,A)')
&    '<StructuredGrid WholeExtent="0 ', NIM-1, ' 0 ', NJM-1, ' 0 0">'
WRITE(20,'(A,I5,A,I5,A)')
&    '<Piece Extent="0 ', NIM-1, ' 0 ', NJM-1, ' 0 0">'

C.....Point Data (coordinates)
WRITE(20,'(A)') '    <Points>'
WRITE(20,'(A)') '    <DataArray type="Float64"
NumberOfComponents="3" '//
&                'Name="Points" format="ascii">'

C.....Write node coordinates (cell centers) - only interior points for
simplicity
DO J = 2, NJM
DO I = 2, NIM
IJ = LI(I + IST) + J
XC_PHYS = XC(IJ)
YC_PHYS = YC(IJ)
WRITE(20,'(3(1PE15.7))') XC_PHYS, YC_PHYS, 0.0D0
ENDDO
ENDDO

WRITE(20,'(A)') '    </DataArray>'
WRITE(20,'(A)') '    </Points>'

C.....Cell Data / Point Data
WRITE(20,'(A)') '    <PointData Scalars="Pressure">'

C.....Pressure
WRITE(20,'(A)') '    <DataArray type="Float64" Name="Pressure"
'//

```

```

&          'format="ascii">'
DO J = 2, NJM
  DO I = 2, NIM
    IJ = LI(I + IST) + J
    WRITE(20,'(1PE15.7)') P(IJ)
  ENDDO
ENDDO
WRITE(20,'(A)') '          </DataArray>'

C.....U velocity
WRITE(20,'(A)') '          <DataArray type="Float64" Name="U" '//
&          'format="ascii">'
DO J = 2, NJM
  DO I = 2, NIM
    IJ = LI(I + IST) + J
    WRITE(20,'(1PE15.7)') U(IJ)
  ENDDO
ENDDO
WRITE(20,'(A)') '          </DataArray>'

C.....V velocity
WRITE(20,'(A)') '          <DataArray type="Float64" Name="V" '//
&          'format="ascii">'
DO J = 2, NJM
  DO I = 2, NIM
    IJ = LI(I + IST) + J
    WRITE(20,'(1PE15.7)') V(IJ)
  ENDDO
ENDDO
WRITE(20,'(A)') '          </DataArray>'

C.....Temperature (if solved)
IF (LCAL(IEN)) THEN
  WRITE(20,'(A)') '          <DataArray type="Float64"
Name="Temperature" '//
&          'format="ascii">'
DO J = 2, NJM
  DO I = 2, NIM
    IJ = LI(I + IST) + J
    WRITE(20,'(1PE15.7)') T(IJ)
  ENDDO
ENDDO
WRITE(20,'(A)') '          </DataArray>'

```

```

ENDIF

C.....Turbulence quantities (if solved)
  IF (LCAL(ITE) .AND. LCAL(IED)) THEN
    WRITE(20,'(A)') '          <dataArray type="Float64" Name="TurbKE"
'//
    &          'format="ascii">'
    DO J = 2, NJM
      DO I = 2, NIM
        IJ = LI(I + IST) + J
        WRITE(20,'(1PE15.7)') TE(IJ)
      ENDDO
    ENDDO
    WRITE(20,'(A)') '          </dataArray>'

    WRITE(20,'(A)') '          <dataArray type="Float64" Name="Omega" '//
    &          'format="ascii">'
    DO J = 2, NJM
      DO I = 2, NIM
        IJ = LI(I + IST) + J
        WRITE(20,'(1PE15.7)') ED(IJ)
      ENDDO
    ENDDO
    WRITE(20,'(A)') '          </dataArray>'

C.....Eddy viscosity
    WRITE(20,'(A)') '          <dataArray type="Float64"
Name="EddyViscosity" '//
    &          'format="ascii">'
    DO J = 2, NJM
      DO I = 2, NIM
        IJ = LI(I + IST) + J
        WRITE(20,'(1PE15.7)') VIS(IJ)
      ENDDO
    ENDDO
    WRITE(20,'(A)') '          </dataArray>'
  ENDIF

  WRITE(20,'(A)') '          </PointData>'

C.....Close tags
  WRITE(20,'(A)') '          </Piece>'
  WRITE(20,'(A)') '          </StructuredGrid>'

```

```

WRITE(20,'(A)') '</VTKFile>'

CLOSE(UNIT=20)

WRITE(2,*) 'VTK file written: ', TRIM(FNAME)
IF (.NOT. LTIME) WRITE(*,*) 'VTK file written: ', TRIM(FNAME)

RETURN
END

```

How to use it

1. **Add the subroutine** to your `user.f` file.
2. **Call it** from `SOUT` (for steady state) or `TOUT` (for unsteady) in `user.f`.

Example for steady state (add to `SOUT` subroutine):

```

fortran
SUBROUTINE SOUT(K)
    INCLUDE 'param.inc'
    INCLUDE 'indexc.inc'
    INCLUDE 'logic.inc'

    CALL WRITE_VTK(K, ICONT)    ! ICONT is the output counter from main
program
    RETURN
END

```

For unsteady flow, you can call it every `N0TT` steps inside the time loop.

Features

- Uses modern **XML VTK StructuredGrid** (`.vts`) format — best for ParaView.
- Writes **cell-centered** data (matches CAFFA's colocated grid).
- Includes: Pressure, U, V, Temperature (if active), k (TurbKE), ω (Omega), Eddy Viscosity.
- Handles axisymmetric case indirectly (coordinates are as stored in `XC` , `YC`).

- ASCII output for easy debugging.

Would you like a version that also writes **velocity magnitude**, **pressure coefficient**, or a **legacy** .vtk version instead?